

By zrealizować zadanie należało pierw dokładnie zrozumieć czym jest drukarka fiskalna, z czym i jak współpracuje, jaka jest jej rola w systemach POS jak i podstawy systemów POS.

Następnie zapoznałem się z modelem którego dotyczy zadanie tj. Deon oraz załączoną dokumentacją.

Z dostępnych aktualnie do zakupu jest wariant Deon Online, więc przyjąłem że chodzi o ten wariant.

W treści zadania podane jest, by skorzystać z protokołu zamieszczonego w załączniku, toteż całe zadanie opiera się na nim.

W załączonej dokumentacji rozpisany jest klasyczny protokół (nie nowszy XML), więc korzystamy z klasycznego.

Drukarka Deon Online potrafi obsługiwać trzy style poleceń – Novitus, Novitus zgodny oraz Posnet.

Jako że domyślnym jest tryb Novitus jak i to on jest opisany w załączonej dokumentacji z takiego też korzysta program.

W treści zadania napisano, że program jako listę parametrów przyjmuje nazwę towaru, cenę, stawkę VAT i opcjonalnie rabat kwotowy.

Napisano nazwa towaru, nie zaś towarów, nie ma też wspomnianej kwestii sztuk, przyjmuję więc że chodzi o jeden towar (pozycję) w ilości sztuk 1.

Zgodnie z dostarczoną dokumentacją pole PTU (stawka VAT) ma typ ZNAK i przyjmuje wartość grupy, więc zakładałem taki format danych wejściowych dla stawki VAT i to że drukarka ma skonfigurowane grupy PTU i użytkownik wie która grupa przypada jakiej stawce.

Przyjęto interfejs szeregowy jako standardowy zgodnie z Opisem interfejsów (pdf strona 295)

Przyjęto parametry połączenia zgodnie z informacjami z tej samej strony dokumentacji.

Protokół zakłada zawsze brak parzystości, 8 bitów i znak stopu, toteż przyjęto te wartości jako stałe.

Podobnie jest z baud, gdzie wartości inne niż 9600 są dostępne tylko dla Quarto/Delio.

Zakładałem handshaking programowy XON-XOFF zgodnie z tabelą 11.1, który działa zawsze, w odróżnieniu od RTS/CTS który wymaga fizycznego połączenia tych sygnałów (lub poprawnej ich emulacji), zaś programowy handshaking działa bezproblemowo zarówno dla RS232 jak i USB.

W zadaniu podano, że program ma być prosty, toteż rozbudowywanie zadania ponad to co polecane miałyby się z poleceniem, toteż pomijam np. odczyt ustawionych stawek PTU czy odczyt błędów zwracanych przez drukarkę.

Po etapie przyjęcia wstępnych założeń potrzebnych do realizacji zadania przeszedłem do następnych etapów.

CMakeFile:

Na początek tworzenia programu założenia projektowe przekazywane do cmake.

By zapewnić że program zostanie poprawnie skompilowany przyjęto wersję CMake co najmniej 3.16, co daje wystarczająco współczesny a zarazem ogólnodostępny wariant.

Ładowany jest tylko język C++ zgodnie z treścią zadania.

Wybrany jest standard C++17, włączony jest wymóg stosowania standardu – w przypadku niemożliwości spełnienia program nie skompiluje się zapobiegając stosowaniu starszych standardów.

Wyłączone są również EXTENSIONS, by zablokować stosowanie rozszerzeń kompilatora, które mogłyby spowodować różnice między stosowanymi kompilatorami.

Kroki te podjęto celem zapewnienia jednolitej odtwarzalności programu i przewidywalnego działania.

W późniejszym etapie po utworzeniu testów rozbudowałem CMakeFile by ułatwić testy.

W protokole istnieją tryby drukowania transakcji/otwierania paragonu związane z ilością pozycji: blokowy gdy znamy z góry ilość pozycji na paragonie oraz on-line gdy tej liczby nie znamy (pdf str.89).

W naszym zadaniu znamy ilość pozycji na paragonie z góry, toteż moglibyśmy zastosować tryb blokowy, w praktyce jednak w zdecydowanej większości przypadków korzysta się z trybu on-line gdy nie jest znana ilość pozycji na paragonie, dlatego też zdecydowałem się wybrać ten tryb.

Rozkaz początek transakcji ma 3 warianty, zależnie od tego jaki typ paragonu chcemy wystawić - zwykły paragon, paragon z dodatkowymi liniami tekstu, czy paragon z NIP / REGON / PESEL

Nie ma komendy która informuje drukarkę jaki typ paragonu będziemy drukować, drukarka sama rozpoznaje typ paragonu po formacie danych przekazanych rozkazem początek transakcji.

Dla zadania prostego programu przyjąłem zwykły paragon – najmniej rozbudowany wariant rozkazu początek transakcji.

By uniemożliwić wystąpienie błędów obliczeniowych przy obliczaniu kwot (dla poprawności jak i by nie być odrzuconym przez drukarkę w przypadku błędu – jak opisuje protokół) zastosowano obliczenia na liczbach całkowitych, w formacie groszy, i to na nich wykonywane są wszelkie kalkulacje, a następnie zamieniane są na liczby w formacie złotych i groszy.

Program jako parametry ma przyjmować tylko nazwę towaru, cenę, stawkę VAT i opcjonalnie rabat kwotowy.

Wynikało by z tego że albo trzeba zapisać w kodzie sztywny port i zmiana portu wymaga rekompilacji programu, co jest niewygodne, albo dodać obsługę przechowywania portu w pliku co jednak nieco komplikuje program, albo dodać możliwość podania portu dodatkowym parametrem.

Jako że program ma być prosty ale nie chce by był niewygodny w obsłudze, zdecydowałem się na kompromis by dodać obsługę portu za pomocą opcjonalnego dodatkowego parametru przekazywanego programowi podczas uruchomienia aplikacji. Jeżeli tego nie zrobimy, przyjmie standardowo /dev/ttyUSB0

Opis użycia interfejsu:

# bez podawania portu – użyje domyślnego /dev/ttyUSB0

./deon\_printer\_app "NAZWA" CENA PTU

./deon\_printer\_app "NAZWA" CENA PTU [RABAT kwotowy]

# z podaniem portu:

./deon\_printer\_app --port=/dev/ttyS0 "NAZWA" CENA PTU

./deon\_printer\_app --port=/dev/ttyS0 "NAZWA" CENA PTU [RABAT kwotowy]

W przypadku problemów z uprawnieniami pliku należy zastosować:

chmod 755 deon\_printer\_app

(każdy czyta, każdy wykonuje, tylko właściciel może zapisać):

Testowanie programu:

Można przetestować program bez drukarki zmieniając wartość DRY\_RUN w pliku main.cpp na true

```
constexpr bool DRY_RUN = true;
```

spowoduje to wypisanie wyjścia nie na port a na konsole, jak i pominie sprawdzanie obecności portu

Program posiada też testy jednostkowe których użycie opisano w README.md

Realizowanie zadania:

Zacząłem od przeczytania polecenia zadania ze zrozumieniem, tak by zrozumieć postawione wymagania i zrozumieć istotę zadania.

Następnie przeszedłem od zebrania wymagań – program w C++ na Linux, uruchamiany z linii komend, który ma wystawić paragon na drukarce Deon Online po RS232/USB CDC, według opisanego protokołu.

Następnie zapoznałem się z dostarczoną dokumentacją, domeną i terminologią:

co to jest drukarka fiskalna, co to jest kasa fiskalna, czym się różnią,

czym są systemy POS i jakie są w nich relacje między ich elementami, jak działają, opisem urządzenia,

Na tej podstawie wypisałem główne zadania: parsowanie argumentów CLI, logika paragonu (cena, PTU, rabat), oraz techniczną komunikację z drukarką po porcie szeregowym i budowę ramek.

Z dokumentacji ustaliłem m.in.

Interfejs komunikacji – RS232 który na linuxie dzięki standardowemu sterownikowi może być używany tak samo jak USB CDC by wiedzieć jak połączyć się z urządzeniem,

Wybrałem handshake softwarowy XON/XOFF jako najbardziej uniwersalny i pewny zarówno dla RS232 jak i USB CDC,

parametry połączenia – baud 9600, 8 bitów, brak parzystości, 1 bit stopu

potrzebne rozkazy, zasady ich działania,

informacje o sumie kontrolnej

oraz to że:

musi obsługiwać zarówno paragon z rabatem kwotowym jak i paragon bez rabatu kwotowego (wtedy nie ma parametru) musi rozpoznawać jaka to wersja po ilości parametrów i wybrać odpowiednią wersję

do tego musi robić pomiary w parsując złotówki i grosze do liczb całkowitych jako grosze, przeliczać jako grosze sztuki i rabat kwotowy (opcjonalny) (sztuk 1 w tym zadaniu i rabat kwotowy)

nie może być błędów w obliczeniach, muszą być jako liczby całkowite jako grosze, a po obliczeniach zamienione z powrotem grosze na złotówki.grosze

musi być zawsze format wyjściowy złotówki.grosze, i taki powinien też być wejściowy, choć obsługiłem np. jedna cyfry groszy (5.2 zamienia w 5.20), ale 5.234 zwraca błąd

musi obliczać CRC zgodnie z opisem w protokole i wstawiać go w miejscach określonych według protokołu z pdfa,

stawkę VAT podaje się według ustawionych w drukarce grup PTU

O architekturze decydowałem ustalając wymagania i biorąc je pod uwagę oraz dane wejściowe i wyjściowe oraz to, co ma stać się po drodze.

Określiłem i spisałem co po kolei ma zrobić program - rozpisanie flow programu, zadania jakie ma realizować.

Spisałem jakie wymagania stawia dokumentacja i protokół, takie jak konieczność obliczania dokładnie by przeszedł paragon – konieczność użycia liczb całkowitych,

Ustalenie poleceń z protokołu, potrzebnych do realizacji zadania,

Rozplanowałem architekturę rozdzielającą program na moduły zgodne z przepływem programu, z tym co po kolei ma robić program.

Rozplanowałem warstwy które powinny tworzyć program,

Najpierw próbowałem samemu na logikę, wiedzę, znajomość zasad solid i intuicję dojść jak najdalej, ustalić wymagania, wejście wyjście, jak najwięcej kroków, co ma zrobić program, jakie dane przyjąć co dokładnie z nimi zrobić, jakie wysłać polecenia, jaki ma być przepływ programu, jak nawiązać połączenie z drukarką i jakie mają być interfejs i parametry połączenia, jak te rzeczy mają wyglądać według protokołu.

Później szukałem w literaturze odwzorowania i podobieństw mojej wizji do ustalonych książkowo opisanych istniejących wzorcach

i dość sporo się pokryło z tym co mi wyszło, dotarłem m.in. do Design Patterns: Elements of Reusable Object-Oriented Software

Odświeżyłem kwestie architektury, wzorców projektowych, oraz zadań do zrobienia przy tym programie i ogólnie jak to powinno się dokładnie książkowo robić według sprawdzonych sposobów postępowania,

Ustalałem architekturę w oparciu o opisane wzorce najbardziej zbliżone do tego jak sam bym to widział.

Z tego ustaliłem architekturę:

**Prosta architektura warstwowa oparta na podejściu Transaction Script, z cienką fasadą nad protokołem i adapterem do warstwy sprzętowej**

Na tej podstawie następnie podzieliłem program na warstwy i ustaliłem, co każda z nich ma zrobić z danymi. Przyjąłem zasadę, że warstwy wyżej korzystają tylko z warstw niżej, a zależności idą w jedną stronę, dzięki czemu logika paragonu jest odseparowana od szczegółów RS232 i Linuxa.

Ustaliłem jakie mechanizmy zastosować, np. RAII dla pewnego uwolnienia zasobów po wywołaniu.

Następnie tworzyłem szkielet, by zacząć tworzyć kod, relacje i przepływ.

Zaczynając od utworzenia plików, ustalenia includów, headerów.

Opis architektury:

Kod został napisany tak jak do tej pory zazwyczaj pisałem, po flow programu, ale stosując kilka klasycznych, prostych wzorców razem:

### 1. Główny styl: Transaction Script

To dokładnie pasuje do przepływu:

main() → wczytaj argumenty → policz kwoty → zbuduj ramki → wyślij je po kolei

U Fowlera to się nazywa **Transaction Script** – jeden prosty „skrypt transakcyjny”, który realizuje konkretny przypadek użycia (tu: *wystaw paragon z jedną pozycją i ewentualnym rabatem*).

Zastosowałem podejście w stylu *Transaction Script*: główna funkcja w uporządkowany sposób realizuje pojedynczy scenariusz biznesowy – wystawienie prostego paragonu – krok po kroku.

---

### 2. Struktura modułów: Layered Architecture

**architektura warstwowa** (Layered Architecture):

- warstwa „wejścia” → main (parsowanie argumentów),
- warstwa „logiki” → receipt\_logic, money\_utils = przeliczanie kwot w groszach,
- warstwa „protokołu” → novitus\_protocol = budowa ciał komend i ramek,
- warstwa „infrastruktury” → SerialPort na termios,

Kod jest podzielony warstwowo: osobno warstwa wejścia (CLI), logika biznesowa (kwoty w groszach), warstwa protokołu drukarki (format ramek, CRC) oraz warstwa infrastruktury (port szeregowy na termios). Dzięki temu main tylko orkiestruje te warstwy.

---

### 3. Prosty „wzorzec” wokół drukarki: Mała Fasada + Adapter

- funkcja printSimpleReceipt(const ReceiptContext&, SerialPort&) działa jak **mała Fasada** na protokół – main nie musi znać \$h/\$l/\$Y/\$e/\$z, tylko woła jedną funkcję,
- klasa SerialPort jest w praktyce **Adapterem** do POSIX-a:
  - ukrywa open/tcsetattr/write,
  - daje proste writeAll.

Dostęp do drukarki owinąłem w prostą klasę SerialPort, która adapteruje niskopoziomowy interfejs POSIX (termios, write) do prostego wywołania writeAll. Na to nałożyłem małą „fasadę” – funkcję printSimpleReceipt, która ukrywa szczegóły protokołu (kolejność komend, budowę ramek) przed resztą aplikacji.

Jak projektowałem ten program:

### 1. **Najpierw zrozumienie branży:**

Zacynałem od zrozumienia domeny: czym jest drukarka fiskalna, jaką pełni rolę w systemie POS i jak działa protokół Novitus na poziomie ramek i komend.

### 2. **Potem rozbicie na odpowiedzialności (warstwy):**

Na tej podstawie podzieliłem program na warstwy:

- main – pobieranie parametrów z linii poleceń,
- logika kwot – parsowanie wartości typu „12.34” do groszy i proste obliczenia,
- protokół drukarki – budowa ciał komend i ramek z CRC,
- infrastruktura – obsługa portu szeregowego na Linuxie (termios).

### 3. **Wybór stylu Transaction Script:**

Główny przepływ w main() zaprojektowałem w stylu Transaction Script: jedna funkcja orkuje kroki w ustalonej kolejności – wczytaj parametry, oblicz kwoty, zbuduj ramki, wyślij je po kolei do drukarki.

### 4. **Cienka fasada + adapter:**

Żeby uprościć użycie protokołu, dodałem małą fasadę `printSimpleReceipt(const ReceiptContext&, SerialPort&)`, która zna kolejność komend i szczegóły protokołu. Dostęp do samego urządzenia opakowałem w klasę `SerialPort`, która pełni rolę adaptera między niskopoziomym POSIX-em (`open`, `termios`, `write`) a prostym wywołaniem `writeAll`.

### 5. **Iteracyjne uzbrajanie warstw:**

Implementację robiłem iteracyjnie: najpierw szkielet z pustymi funkcjami i fake’owym `SerialPort` wypisującym na `stdout`, potem dołożyłem parsowanie kwot do groszy, później budowę ramek i CRC, na końcu prawdziwe ustawienia portu szeregowego i testy na wirtualnym porcie.

Program był tworzony w duchu zasad SOLID jak i uwzględniając takie kwestie jak wycieki pamięci.

Na sam koniec w ramach ćwiczenia self code review dodawałem komentarze do każdej linii – mam świadomość że jest ich nadmiar i normalnie stosuje ich o wiele mniej, pierwotnie dałem ich sporo mniej, ale jest to głównie efekt ćwiczenia dla mnie a przy okazji mogą przyspieszyć zrozumienie programu więc je zostawiłem.

Eksperymentowałem z ich położeniem w kodzie by nie utrudniały czytelności kodu a zarazem dostarczały informacji. Jako że zadanie ma znamiona ćwiczenia uznałem że jest to w tym przypadku akceptowalne choć przyznam że wielokrotnie zastanawiałem się czy ich nie zredukować, jednak uznałem że w przypadku ćwiczenia jest to akceptowalne.

## **Sekwencja poleceń protokołu dla jednego towaru z opcjonalnym rabatem kwotowym od całego paragonu**

### **1. 3.4.1. Początek transakcji – paragon on-line**

**Rozkaz:** 0\$h – pierwszy wariant struktury parametrów

Znaczenie: otwarcie paragonu w trybie on-line (ilość pozycji = 0) czyli ON-LINE.

### **2. 3.4.2 Linia paragonu - wystawienie pozycji paragonu (jeden towar, ilość 1)**

**Rozkaz:** 1\$I – pierwszy wariant struktury parametrów

Pola:

- numer linijki = 1,
- nazwa towaru = parametr NAZWA,
- ilość = 1,
- PTU = parametr PTU (A..G),
- cena = parametr CENA,
- brutto = cena \* ilość (dla ilości 1 równe CENA).

Wariant bez rabatu pozycji.

### **3. 3.4.15. Rabat/narzut do paragonu od podsumy - opcjonalny rabat kwotowy od całego paragonu (od podsumy)**

**Rozkaz:** \$Y – „Rabat/narzut do paragonu od podsumy”, Rodzaj rabatu = 3 (rabat kwotowy)

Pola:

- rodzaj rabatu = 3,
- numer opisu = np. 0 (bez predefiniowanego opisu),
- podsuma = wartość brutto wszystkich pozycji przed rabatem (dla jednego towaru = brutto z 1\$I),
- rabat = kwota rabatu podana jako opcjonalny parametr programu,
- opis rabatu – opcjonalny tekst (np. „RABAT PARAGONU”) lub pusty.

### **4. 3.4.9. Standardowe zatwierdzenie transakcji - (zamknięcie paragonu)**

**Rozkaz:** 1\$e – „Standardowe zatwierdzenie transakcji”

Pola:

- akcja = 1 (zatwierdzenie),
- rabat = 0 (rabat rozliczony wcześniej przez \$Y),
- nazwa kasjera – stały lub pusty ciąg,
- wpłata = kwota do zapłaty po rabacie (podsuma – rabat kwotowy),
- razem = ta sama kwota co wpłata.

Wszystkie powyższe rozkazy są przesyłane w ramach protokołu (nagłówek <ESC>P, pola rozdzielone separatorami, bajt kontrolny XOR, zakończenie <ESC>\).